

# CMSC 25300 / 35300

## Homework 4

For all the coding questions, please show the outputs of your code.

1. **Gram-Schmidt** (24 points). In this problem, we want to perform the Gram-Schmidt algorithm and evaluate its performance on the Hilbert matrix with order 7 given as below.

$$X = \begin{bmatrix} 1 & \frac{1}{2} & \frac{1}{3} & \frac{1}{4} & \frac{1}{5} & \frac{1}{6} & \frac{1}{7} \\ \frac{1}{2} & \frac{1}{3} & \frac{1}{4} & \frac{1}{5} & \frac{1}{6} & \frac{1}{7} & \frac{1}{8} \\ \frac{1}{3} & \frac{1}{4} & \frac{1}{5} & \frac{1}{6} & \frac{1}{7} & \frac{1}{8} & \frac{1}{9} \\ \frac{1}{4} & \frac{1}{5} & \frac{1}{6} & \frac{1}{7} & \frac{1}{8} & \frac{1}{9} & \frac{1}{10} \\ \frac{1}{5} & \frac{1}{6} & \frac{1}{7} & \frac{1}{8} & \frac{1}{9} & \frac{1}{10} & \frac{1}{11} \\ \frac{1}{6} & \frac{1}{7} & \frac{1}{8} & \frac{1}{9} & \frac{1}{10} & \frac{1}{11} & \frac{1}{12} \\ \frac{1}{7} & \frac{1}{8} & \frac{1}{9} & \frac{1}{10} & \frac{1}{11} & \frac{1}{12} & \frac{1}{13} \end{bmatrix}$$

- a. (8 points) Revisiting the Gram-Schmidt algorithm in the lecture 6, write your own code to perform Gram-Schmidt orthogonalization.

Here is some code to get you started:

```
import numpy as np

def gram_schmidt(X):
    # X is an n-by-p matrix.
    # Returns U an orthonormal matrix.
    # eps is a threshold value to identify if a vector
    # is nearly a zero vector.
    eps = 1e-12

    n, p = X.shape
    U = np.zeros((n, 0))
    for j in range(p):
        # Get the j-th column of matrix X
        v = X[:, j]
        # Write your own code here: Perform the
        # orthogonalization by subtracting the projections on
        # all columns of U. And then check whether the vector
        # you get is nearly a zero vector.

    return U
```

- b. (8 points) Use the function you obtained and perform Gram-Schmidt orthogonalization on the matrix  $X$ . Evaluate the accuracy by checking the orthogonality of the resulting

basis. Report the  $L_1$  matrix norm of the error matrix you get (Note:  $L_1$  norm of a matrix  $A$  is  $\sum_{i,j} |A_{ij}|$ ).

Instead of manually typing the values of  $X$ , here is some code that you could utilize for generating the order 7 Hilbert matrix.

```
def hilbert_matrix(n):
    X = np.array([[1.0 / (i + j - 1) for i in \
range(1, n + 1)] for j in range(1, n + 1)])
    return X
```

- c. (8 points) **Modified Gram-Schmidt.** Use the provided code of the modified Gram-Schmidt to perform the orthogonalization on the matrix  $X$ . Assess the accuracy of the resulting orthogonal basis and compare it with the results obtained in part (b). Discuss the distinctions between the two implementations.

```
def modified_gram_schmidt(X):
    # Define a threshold value to identify if a vector
    # is nearly a zero vector.
    eps = 1e-12

    n, p = X.shape
    U = np.zeros((n, 0))

    for j in range(p):
        # Get the j-th column of matrix X
        v = X[:, j]
        for i in range(j):
            # Compute and subtract the projection of
            # vector v onto the i-th column of U
            v = v - np.dot(U[:, i], v) * U[:, i]
        v = np.reshape(v, (-1, 1))
        # Check whether the vector we get is nearly
        # a zero vector
        if np.linalg.norm(v) > eps:
            # Normalize vector v and append it to U
            U = np.hstack((U, v / np.linalg.norm(v)))

    return U
```

2. (20 points) **The SVD.** Imagine there are three points in  $\mathbb{R}^2$ , where

$$\mathbf{x}_1 = \begin{bmatrix} \sqrt{2} \\ 0 \end{bmatrix}, \quad \mathbf{x}_2 = \begin{bmatrix} 3 \\ 2 \end{bmatrix}, \quad \mathbf{x}_3 = \begin{bmatrix} 0 \\ \sqrt{7} \end{bmatrix}.$$

Let data matrix be  $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3] \in \mathbb{R}^{2 \times 3}$ .

- a. (5 points) Find an **orthonormal basis** for the span of the three points.
  - b. For any vector  $\mathbf{v}$  in  $\mathbb{R}^2$ , we can regress all points onto  $\mathbf{v}$ , i.e., project all data points onto the subspace  $\mathbf{V}$  spanned by  $\mathbf{v}$ .
    - (i) (3 points) Write an expression for the projection matrix  $\mathbf{P}$  onto  $\mathbf{V}$  in terms of  $\mathbf{v}$ .
    - (ii) (3 points) What is the squared distance of each data point from subspace  $\mathbf{V}$ , in terms of  $\mathbf{x}_i$  and  $\mathbf{P}$ ?
    - (iii) (4 points) Now find  $\mathbf{v}$  with  $\|\mathbf{v}\|_2 = 1$  that minimizes sum of the squared distance for all data points. Is  $\mathbf{v}$  unique? Is  $\mathbf{P}$  unique?
  - c. (5 points) Suppose now we are doing full SVD of  $\mathbf{X}$ , i.e.,  $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$ . Provide possible matrices  $\mathbf{U}$  and  $\mathbf{\Sigma}$ . **Hint: use the results of part (b).**
3. (10 points) Let  $\mathbf{X}$  be an  $n \times d$  matrix with right singular vectors  $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_r$ , left singular vectors  $\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_r$ , and corresponding singular values  $\sigma_1, \sigma_2, \dots, \sigma_r$ .
- a. (4 points) Given  $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$ , express the matrix  $\mathbf{U}\mathbf{\Sigma}$  using the singular vectors and singular values above.
  - b. (3 points) Using your answer from part (a), decompose  $\mathbf{X}$  into a sum of rank one matrices in terms of the singular vectors and singular values above.
  - c. (3 points) Recall rank- $k$  approximations  $\mathbf{X}_k$ , where  $k < r$ . Express  $\mathbf{X}_k$  with the sum of rank one matrices.
4. (5 points) Suppose we have a square matrix  $\mathbf{X} = \begin{bmatrix} 1 & 0 \\ 0 & 5 \end{bmatrix}$ . What is its SVD?
5. (5 points) Suppose we have a square matrix  $\mathbf{X} = \begin{bmatrix} -6 & 0 \\ 0 & -2 \end{bmatrix}$ . What is its SVD?
6. (36 points) Consider the smiling face classification problem from HW 2 (dataset can be downloaded from HW2). Design and compare the performances of the classifiers proposed in **a** and **b**, below. In each case, divide the dataset into 8 equal sized subsets (e.g., examples 1–16, 17–32, etc). Use 6 sets of the data to estimate  $\mathbf{w}$  for each choice of the *regularization parameter*, select the best value for the regularization parameter by estimating the error on one of the two remaining sets of data, and finally use the  $\mathbf{w}$  corresponding to the best value of the regularization parameter to predict the labels of the remaining “hold-out” set. Compute the number of mistakes made on this hold-out set and divide that number by 16 (the size of the set) to estimate the error rate. Repeat this process 56 times (for the  $8 \times 7$  different choices of the sets used to select the regularization parameter and estimate the error rate) and average the error rates to obtain a final estimate.
- a. (12 points) Truncated SVD solution. Use the pseudo-inverse  $\mathbf{V}\mathbf{\Sigma}_k^+\mathbf{U}^T$ , where  $\mathbf{\Sigma}_k^+$  is computed by inverting the  $k$  largest singular values and setting others to zero. Here,  $k$  is the regularization parameter and it takes values  $k = 1, 2, \dots, 9$ ; i.e., compute 9 different solutions,  $\hat{\mathbf{w}}_k$ .

- b. (12 points) Regularized LS. Let  $\hat{\mathbf{w}}_\lambda = \arg \min_{\mathbf{w}} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \lambda \|\mathbf{w}\|_2^2$ , for the following values of the regularization parameter  $\lambda = 0, 2^{-1}, 2^0, 2^1, 2^2, 2^3$ , and  $2^4$ . Show that  $\hat{\mathbf{w}}_\lambda$  can be computed using the SVD and use this fact in your code.

Here is some code to get you started:

```
import numpy as np
import matplotlib.pyplot as plt
import scipy.io as sio
import sys

d = sio.loadmat('face_emotion_data.mat')
X = d['X']
y = d['y']

n, p = np.shape(X)

# error rate for regularized least squares
error_RLS = np.zeros((8, 7))
# error rate for truncated SVD
error_SVD = np.zeros((8, 7))

# SVD parameters to test
k_vals = np.arange(9) + 1
param_err_SVD = np.zeros(len(k_vals))

# RLS parameters to test
lambda_vals = np.array([0, 0.5, 1, 2, 4, 8, 16])
param_err_RLS = np.zeros(len(lambda_vals))
```

- c. (12 points) Use the original dataset to generate 3 new features for each face, as follows. Take the 3 new features to be a random linear combination of the original 9 features. This can be done for instance with the Python command `X @ np.random.rand(9,3)` and augmenting the original matrix  $\mathbf{X}$  with the resulting 3 columns. Will these new features be helpful for classification? Why or why not? Repeat the experiments in (a) and (b) above using the 12 features.